

UNIT I:

Operating Systems Overview: Definition, Evolution of O.S, Components & Services of OS, Structure, Architecture, types of Operating Systems, Batch Systems, Concepts of Multiprogramming and Time Sharing, Parallel, Distributed and real time Systems.

Operating Systems Structures: Operating system services and systems calls, system programs, operating system structure, operating systems generations.

1. What is an Operating System?

- A program that acts as an interface between a user of a computer and the computer hardware.
- An operating system acts as the manager of all the resources, i.e. acting as a resource manager
- An operating system is similar to a government. Like a government, it performs no useful function by itself. It simply provides an environment within which other programs can do useful work.
- **Operating system goals:**
 - Execute user programs and make solving user problems easier
 - Make the computer system convenient to use
 - Use the computer hardware in an efficient manner

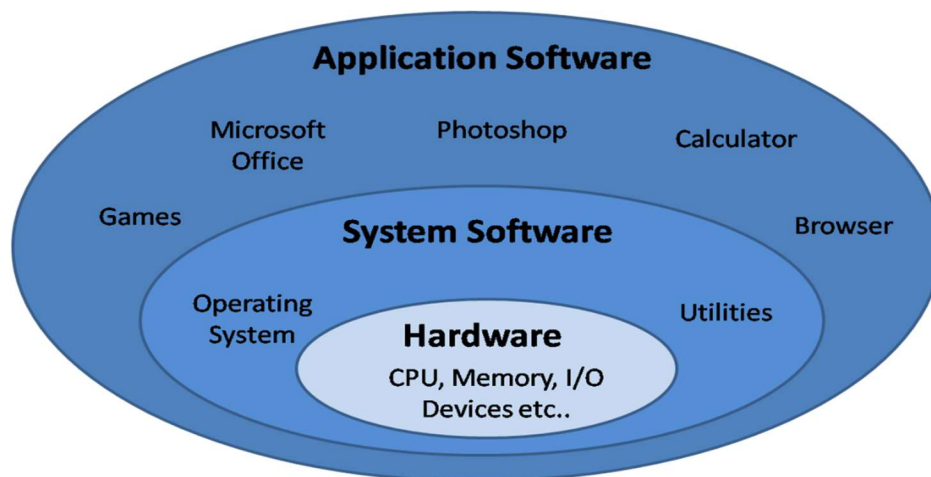


Fig1: Computer System Layers

2. Evolution/Generation of Operating Systems

- **1940s-1950s: Early Beginnings**

- Computers operated without operating systems (OS).
- Programs were manually loaded and run, one at a time.
- The first operating system was introduced in 1956. It was a batch processing system GM-NAA I/O (1956) that automated job handling.

- **1960s: Multiprogramming and Timesharing**

- Introduction of multiprogramming to utilize CPU efficiently.
- Timesharing systems, like CTSS (1961) and Multics (1969), allowed multiple users to interact with a single system.

- **1970s: Unix and Personal Computers**

- Unix (1971) revolutionized OS design with simplicity, portability, and multitasking.

- **1980s: GUI and Networking**

- Graphical User Interfaces (GUIs) gained popularity with systems like Apple Macintosh (1984) and Microsoft Windows (1985).
- Networking features, like TCP/IP in Unix, became essential.

- **1990s: Linux and Advanced GUIs**

- Linux (1991) introduced open-source development.
- Windows and Mac OS refined GUIs and gained widespread adoption.

- **2000s-Present: Mobility and Cloud**

- Mobile OSs like iOS (2007) and Android (2008) dominate.
- Cloud-based and virtualization technologies reshape computing, with OSs like Windows Server and Linux driving innovation.

- **AI Integration – (Ongoing)**

- With the growth of time, Artificial intelligence came into picture. Operating system integrates features of AI technology like Siri, Google Assistant, and Alexa and became more powerful and efficient in much way.
- These AI features with operating system create an entire new feature like voice commands, predictive text, and personalized recommendations.

3. Components & Services of OS

Core Components

An OS is made up of several core components that help manage hardware and software efficiently.

- **Kernel**

- The core of the OS that directly interacts with hardware.
- Manages CPU, memory, and device operations.

- **Process Management**

- Controls execution of programs and multitasking.
- Uses scheduling algorithms like FIFO, Round Robin, etc.

- **Memory Management**

- Allocates and deallocates memory to running processes.
- Uses techniques like paging and segmentation.

- **File System Management**

- Organizes, stores, and retrieves files.
- Supports file systems like NTFS, FAT32, etc.

- **Device Management**

- Manages input/output devices (printers, keyboards, etc.).

- Uses device drivers to enable communication.
- **Security & Access Control**
 - Protects system resources from unauthorized access.
 - Uses authentication methods like passwords and encryption.

Services Provided by an OS

The OS offers several essential services to users and applications.

- **Program Execution**
 - Loads programs into memory and runs them efficiently.
- **I/O Operations**
 - Manages input and output requests from hardware devices.
- **File System Operations**
 - Creates, reads, writes, and deletes files.
- **Communication**
 - Enables **Inter-Process Communication (IPC)** and networking.
- **Error Detection & Handling**
 - Detects system errors and logs them for troubleshooting.
- **Resource Allocation**
 - Assigns CPU, memory, and devices to different programs.
- **Security & Protection**
 - Implements **firewalls, access permissions, and encryption.**

4. OS Structure/Architecture

In operating system Architecture, we've two major terms which define the major components of the operating systems.

- **Kernal**

- Kernal is the central component of operating system architecture.
- A kernal is responsible for all major operations and interaction with the hardware.
- A kernal manages memory, processor, input/output devices and provides interface to application programs to interact with hardware components.

- **Shell**

- Shell is an interface of an operating system. It can be command line interface or a graphical user interface.
- User interacts with an operating system using shell.
- Application programs can also use shell interface to interact with underlying operating system.

- **System Softwares**

- System softwares are the programs which interact with Kernal and provides interface for security managment, memory management and other low level activities.

- **Application Programs**

- Application softwares / Programs are the one using which a user interacts with the operating system to execute a application.
- For example a word processor to create a document and save it on the file system or a notepad to create notes etc.

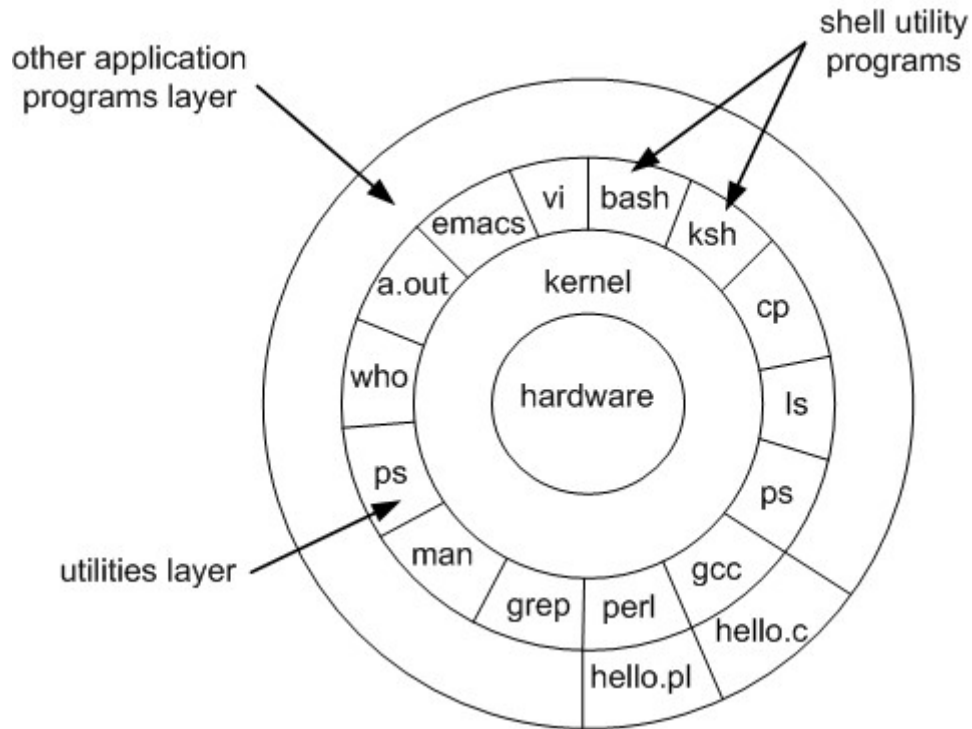


Fig2: Operating System Components

The operating system can be implemented with the help of various structures. The structure of the OS depends mainly on how the various standard components of the operating system are interconnected and merge into the kernel. It is like the blueprint of how an OS is organized and how its different parts interact with each other.

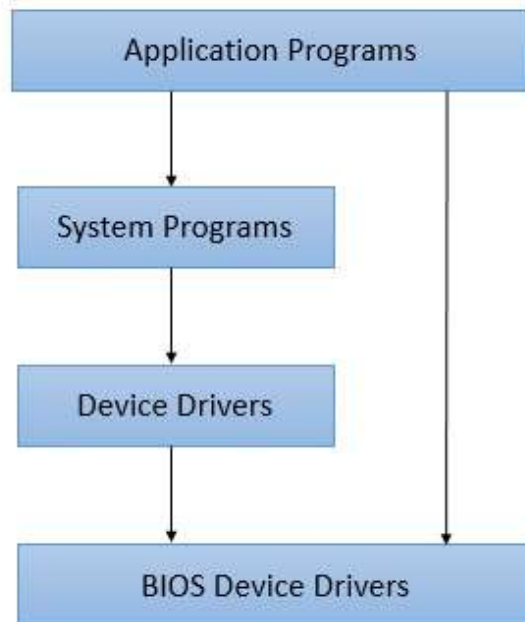
Following are various popular implementations of Operating System structures.

- Simple Structure
- Monolith Structure
- Micro-Kernel Structure
- Exo-Kernel Structure
- Layered Structure
- Modular Structure
- Virtual Machines

4.1 Simple Structure

Simple structure operating systems do not have well-defined structures and are small, simple, and limited. The interfaces and levels of functionality are not well separated. MS-DOS is an example of such an operating system. In MS-DOS, application

programs are able to access the basic I/O routines. These types of operating systems cause the entire system to crash if one of the user programs fails.



Advantages

- **Better Performance** - It delivers better application performance because of the few interfaces between the application program and the hardware.

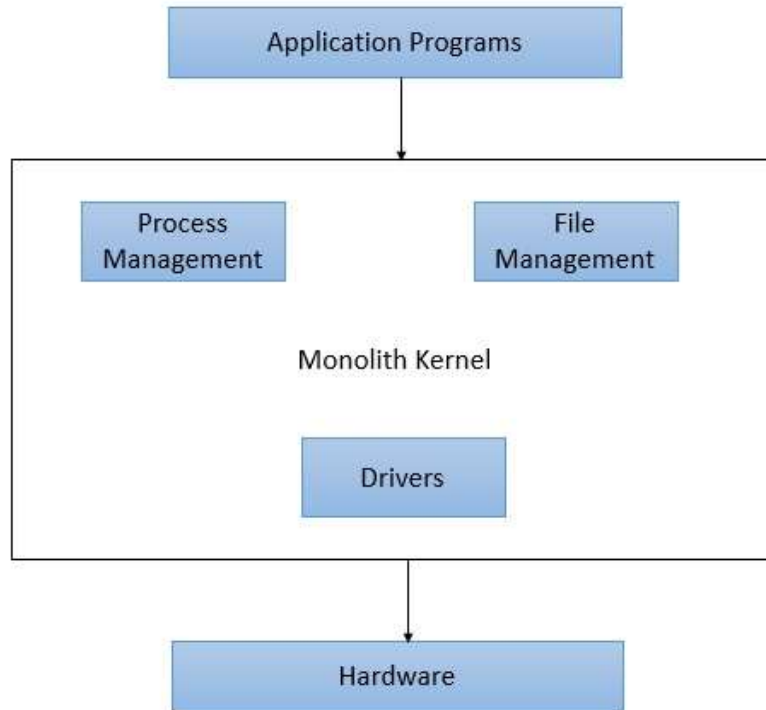
Disadvantages

- **Frequent System Failures** - Being poorly designed, such a system is not robust. If one program fails, entire operating system crashes.

4.2 Monolith Structure

In monolith structured operating system, a central piece of code called kernel is responsible for all major operations of an operating system like file management, memory management, device management and so on.

The kernel is the main component of an operating system and it provides all the services of an operating system to the application programs and system programs.



Advantages

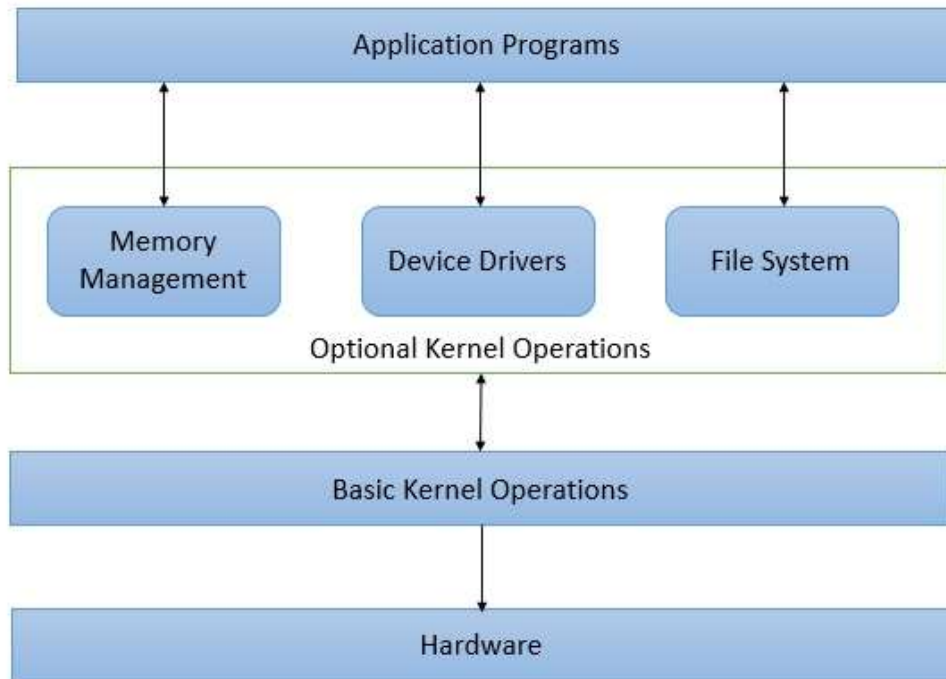
- **Easy Development** - As kernel is the only layer to develop with all major functionalities, it is easier to design and develop.
- **Performance** - As Kernel is responsible for memory management, other operations and have direct access to the hardware, it performs better.

Disadvantages

- **Crash Prone** - As Kernel is responsible for all functions, if one function fails entire operating system fails.
- **Difficult to enhance** - It is very difficult to add a new service without impacting other services of a monolith operating system.

4.3 Micro-Kernel Structure

As in case monolith structure, there was single kernel, in micro-kernel, we have multiple kernels each one specialized in particular service. Each microkernel is developed independent to the other one and makes system more stable. If one kernel fails the operating system will keep working with other kernel's functionalities.



Advantages

- **Reliable and Stable** - As multiple kernels are working simultaneously, chances of failure of operating system is very less. If one functionality is down, operating system can still provide other functionalities using stable kernels.
- **Maintainability** - Being small sized kernels, code size is maintainable. One can enhance a microkernel code base without impacting other microkernel code base.

Disadvantages

- **Complex to Design** - Such a microkernel based architecture is difficult to design.
- **Performance Degradation** - Multi kernel, Multi-modular communication may hamper the performance as compared to monolith architecture.

4.4 Exo-Kernel Structure

Exo-Kernal Structured operating system was designed and developed at MIT. The aim of this design was to keep Kernel size minimal while allowing the application programs to manage hardware resources directly. The purpose of removing abstraction of operating system for hardware resources was to enable application programmer to write high performance code while exo-kernel handles other operations.

Advantages

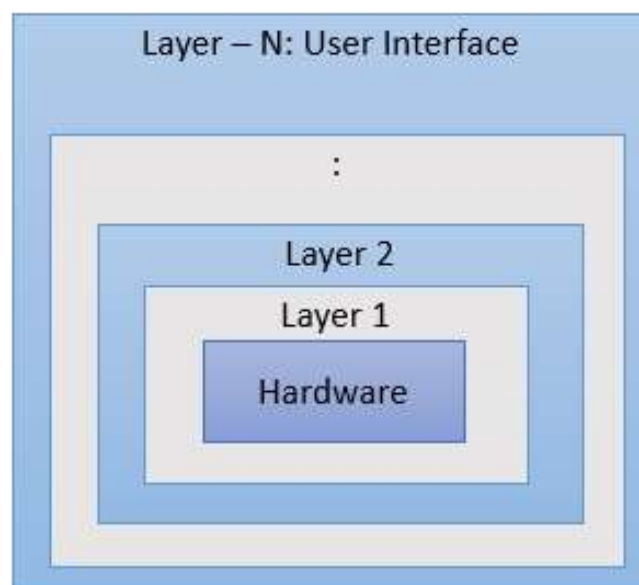
- **High Performance** - As application program can allocate memory, a better designed code can make optimal use and perform better.
- **Application Control** - As resource management is not secured by operating system, application program has more control over system resources and can write custom operations on system resources.

Disadvantages

- **Unreliable and Unsafe** - As security is in application program level, a poorly written code can ruin the system.
- **Complex Design** - Exo-Kernel designing is complicated.

4.5 Layered Structure

An OS can be broken into pieces and retain much more control over the system. In this structure, the OS is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware, and the topmost layer (layer N) is the user interface. These layers are so designed that each layer uses the functions of the lower-level layers. This simplifies the debugging process, if lower-level layers are debugged and an error occurs during debugging, then the error must be on that layer only, as the lower-level layers have already been debugged. The main disadvantage of this structure is that at each layer, the data needs to be modified and passed on which adds overhead to the system. Moreover, careful planning of the layers is necessary, as a layer can use only lower-level layers. UNIX is an example of this structure.



Advantages

- **High Customizable** - Being layered, each layer implementation can be customized easily. A new functionality can be added without impacting other modules as well.
- **Verifiable** - Being modular, each layer can be verified and debugged easily.

Disadvantages

- **Less Performance** - A layered structured operating system is less performance as compared to basic structured operating system.
- **Complex designing** - Each layer is to planned carefully as each layer communicates with lower layer only and a good design process is required to create a layered operating system.

4.6 Modular Structure

Modular structure operating system works on the similar principle as a monolith but with better design. A central kernel is responsible for all major operations of operating system. This kernel has set of core functionality and other services are loaded as modules dynamically to the kernel at boot time or at runtime. Sun Solaris OS is one of the example of Modular structured operating system.

Advantages

- **High Customizable** - Being modular, each module implementation can be customized easily. A new functionality can be added without impacting other modules as well.
- **Verifiable** - Being modular, each layer can be verified and debugged easily.

Disadvantages

- **Less Performance** - A modular structured operating system is less performance as compared to basic structured operating system.
- **Complex designing** - Each module is to planned carefully as each module communicates with kernel. A communication API is to be devised to facilitate the communication.

4.7 Virtual Machine Structure

In this kind of structure, hardware like CPU, memory, hard disks are abstracted into virtual machines. User can use them with actually configure them using execution contexts. Virtual machine takes a good amount of disk space and is to be provisioned. Multiple virtual machines can be created on a single physical machine.

Advantages

- **High Customizable** - Being virtual, functionality are easily accessible, can be customized on need basis.
- **Secure** - Being virtual, and no direct hardware access, such systems are highly secured.

Disadvantages

- **Less Performance** - A virtual structured operating system is less performance as compared to modular structured operating system.
- **Complex designing** - Each virtual component of the machine is to plan carefully as each component is to abstract underlying hardware.

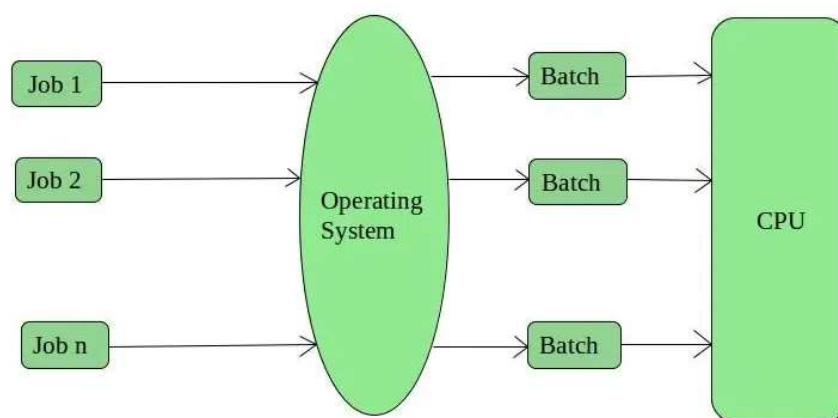
5. Types of Operating Systems

Operating Systems can be categorized according to different criteria like whether an operating system is for mobile devices (examples Android and iOS) or desktop (examples Windows and Linux). Also it can be classified based on functionality basis.

On functionalities basis an operating system area classified as

5.1 Batch Operating System

This type of operating system does not interact with the computer directly. There is an operator which takes similar jobs having the same requirements and groups them into batches. Batch Operating System is designed to manage and execute a large number of jobs efficiently by processing them in groups.



Advantages of Batch Operating System

- Multiple users can share the batch systems.
- The idle time for the batch system is very less.
- It is easy to manage large work repeatedly in batch systems.

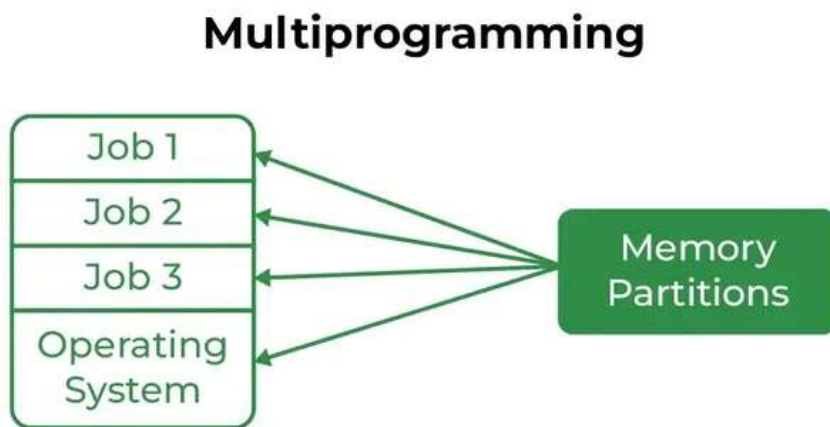
Disadvantages of Batch Operating System

- CPU is not used efficiently. When the current process is doing IO, CPU is free and could be utilized by other processes waiting.
- The other jobs will have to wait for an unknown time if any job fails.
- In batch operating system, average response time increases as all processes are processed one by one.

Examples of Batch Operating Systems: Payroll Systems, Bank Statements, etc.

5.2 Multi-Programming Operating System

Multiprogramming Operating Systems can be simply illustrated as more than one program is present in the main memory and any one of them can be kept in execution. This is basically used for better utilization of resources.

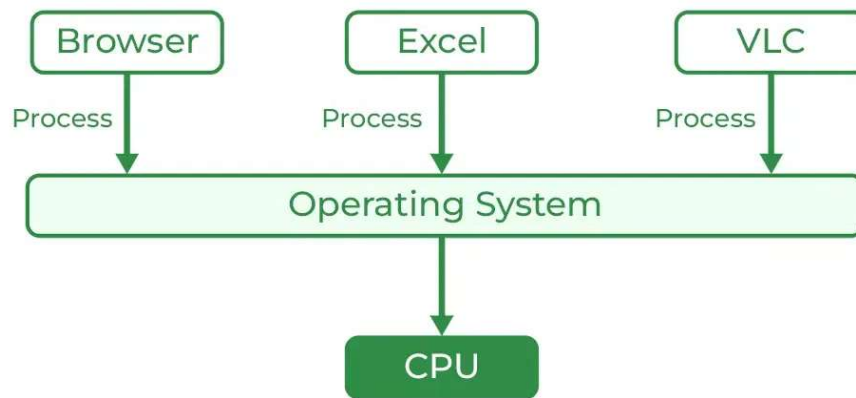


Advantages of Multi-Programming Operating System

- CPU is better utilized and overall performance of the system improves.
- It helps in reducing the response time.

5.3 Multi-Tasking/Time-sharing Operating systems

It is a type of Multiprogramming system with every process running in round robin manner. Each task is given some time to execute so that all the tasks work smoothly. Each user gets the time of the CPU as they use a single system. These systems are also known as Multitasking Systems. The task can be from a single user or different users also. The time that each task gets to execute is called quantum. After this time interval is over OS switches over to the next task.



Advantages of Time-Sharing OS

- Each task gets an equal opportunity.
- CPU idle time can be reduced.
- Resource Sharing: Time-sharing systems allow multiple users to share hardware resources such as the CPU, memory, and peripherals, reducing the cost of hardware and increasing efficiency.
- Improved Productivity: Time-sharing allows users to work concurrently, thereby reducing the waiting time for their turn to use the computer. This increased productivity translates to more work getting done in less time.
- Improved User Experience: Time-sharing provides an interactive environment that allows users to communicate with the computer in real time, providing a better user experience than batch processing.

Disadvantages of Time-Sharing OS

- One must have to take care of the security and integrity of user programs and data.
- Data communication problem.
- High Overhead: Time-sharing systems have a higher overhead than other operating systems due to the need for scheduling, context switching, and other overheads that come with supporting multiple users.

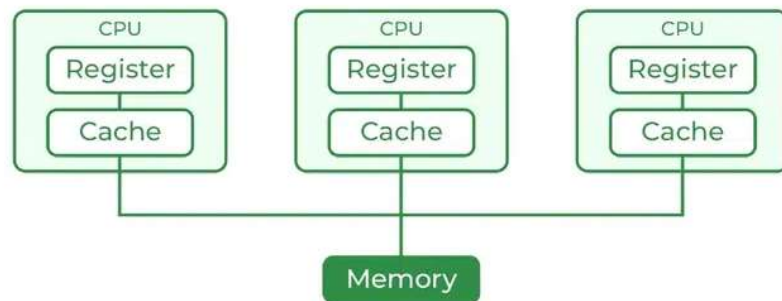
- Complexity: Time-sharing systems are complex and require advanced software to manage multiple users simultaneously. This complexity increases the chance of bugs and errors.
- Security Risks: With multiple users sharing resources, the risk of security breaches increases. Time-sharing systems require careful management of user access, authentication, and authorization to ensure the security of data and software.

Examples of Time-Sharing OS with explanation

- IBM VM/CMS : IBM VM/CMS is a time-sharing operating system that was first introduced in 1972. It is still in use today, providing a virtual machine environment that allows multiple users to run their own instances of operating systems and applications.
- TSO (Time Sharing Option): TSO is a time-sharing operating system that was first introduced in the 1960s by IBM for the IBM System/360 mainframe computer. It allowed multiple users to access the same computer simultaneously, running their own applications.
- Windows Terminal Services: Windows Terminal Services is a time-sharing operating system that allows multiple users to access a Windows server remotely. Users can run their own applications and access shared resources, such as printers and network storage, in real-time.

5.4 Multi-Processing Operating System

Multi-Processing Operating System is a type of Operating System in which more than one CPU is used for the execution of processes. It better the throughput of the System.

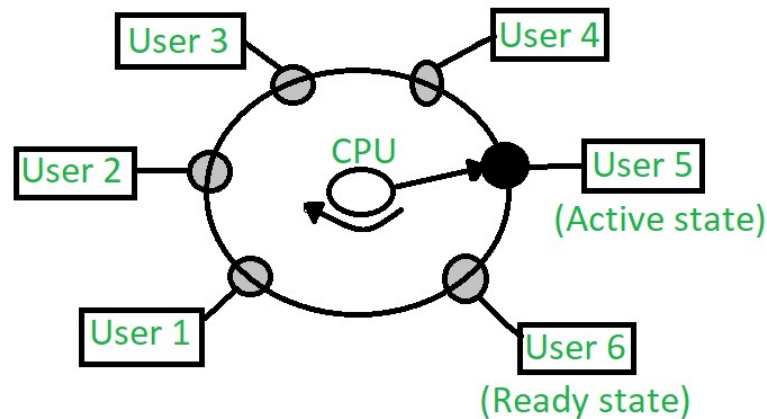


Advantages of Multi-Processing Operating System

- It increases the throughput of the system as processes can be parallelized.
- As it has several processors, so, if one processor fails, we can proceed with another processor.

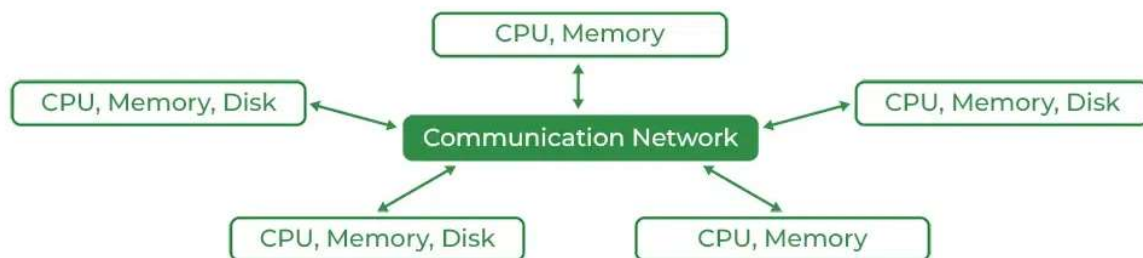
5.5 Multi User Operating Systems

These systems allow multiple users to be active at the same time. This system can be either multiprocessor or single processor with interleaving.



5.6 Distributed Operating System

These types of operating system are a recent advancement in the world of computer technology and are being widely accepted all over the world and, that too, at a great pace. Various autonomous interconnected computers communicate with each other using a shared communication network. Independent systems possess their own memory unit and CPU. These are referred to as loosely coupled systems or distributed systems. These systems' processors differ in size and function. The major benefit of working with these types of the operating system is that it is always possible that one user can access the files or software which are not actually present on his system but some other system connected within this network i.e., remote access is enabled within the devices connected in that network.



Advantages of Distributed Operating System

- Failure of one will not affect the other network communication, as all systems are independent of each other.

- Electronic mail increases the data exchange speed.
- Since resources are being shared, computation is highly fast and durable.
- Load on host computer reduces.
- These systems are easily scalable as many systems can be easily added to the network.
- Delay in data processing reduces.

Disadvantages of Distributed Operating System

- Failure of the main network will stop the entire communication.
- To establish distributed systems the language is used not well-defined yet.

Issues with Distributed Operating Systems

- Networking causes delays in the transfer of data between nodes of a distributed system. Such delays may lead to an inconsistent view of data located in different nodes, and make it difficult to know the chronological order in which events occurred in the system.
- Control functions like scheduling, resource allocation, and deadlock detection have to be performed in several nodes to achieve computation speedup and provide reliable operation when computers or networking components fail.
- Messages exchanged by processes present in different nodes may travel over public networks and pass through computer systems that are not controlled by the distributed operating system.

5.7 Network Operating System

These systems run on a server and provide the capability to manage data, users, groups, security, applications, and other networking functions. These types of operating systems allow shared access to files, printers, security, applications, and other networking functions over a small private network. One more important aspect of Network Operating Systems is that all the users are well aware of the underlying configuration, of all other users within the network, their individual connections, etc. and that's why these computers are popularly known as tightly coupled systems.



Advantages of Network Operating System

- Highly stable centralized servers.
- Security concerns are handled through servers.
- New technologies and hardware up-gradation are easily integrated into the system.
- Server access is possible remotely from different locations and types of systems.

Disadvantages of Network Operating System

- Servers are costly.
- User has to depend on a central location for most operations.
- Maintenance and updates are required regularly.

Examples of Network Operating Systems are Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, BSD, etc.

5.8 Real-Time Operating System

These types of OSs serve real-time systems. The time interval required to process and respond to inputs is very small. This time interval is called **response time**. **Real-time systems** are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Types of Real-Time Operating Systems

Hard Real-Time Systems: Hard Real-Time OSs are meant for applications where time constraints are very strict and even the shortest possible delay is not acceptable. These

systems are built for saving life like automatic parachutes or airbags which are required to be readily available in case of an accident.

Soft Real-Time Systems: These OSs are for applications where time-constraint is less strict.

Advantages of RTOS

- Task Shifting: The time assigned for shifting tasks in these systems is very less. For example, in older systems, it takes about 10 microseconds in shifting from one task to another, and in the latest systems, it takes 3 microseconds.
- Focus on Application: Focus on running applications and less importance on applications that are in the queue.
- Real-time operating system in the embedded system: Since the size of programs is small, RTOS can also be used in embedded systems like in transport and others.
- Error Free: These types of systems are error-free.

Disadvantages of RTOS

- Limited Tasks: Very few tasks run at the same time and their concentration is very less on a few applications to avoid errors.
- Complex Algorithms: The algorithms are very complex and difficult for the designer to write on.
- Device driver and interrupt signals: It needs specific device drivers and interrupts signal to respond earliest to interrupts.

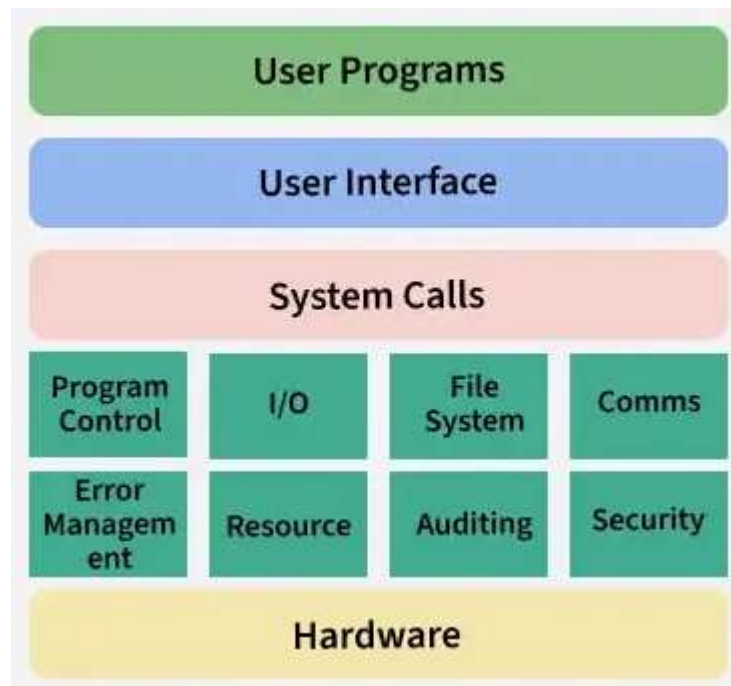
Examples of Real-Time Operating Systems are Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

5.9 Mobile Operating Systems

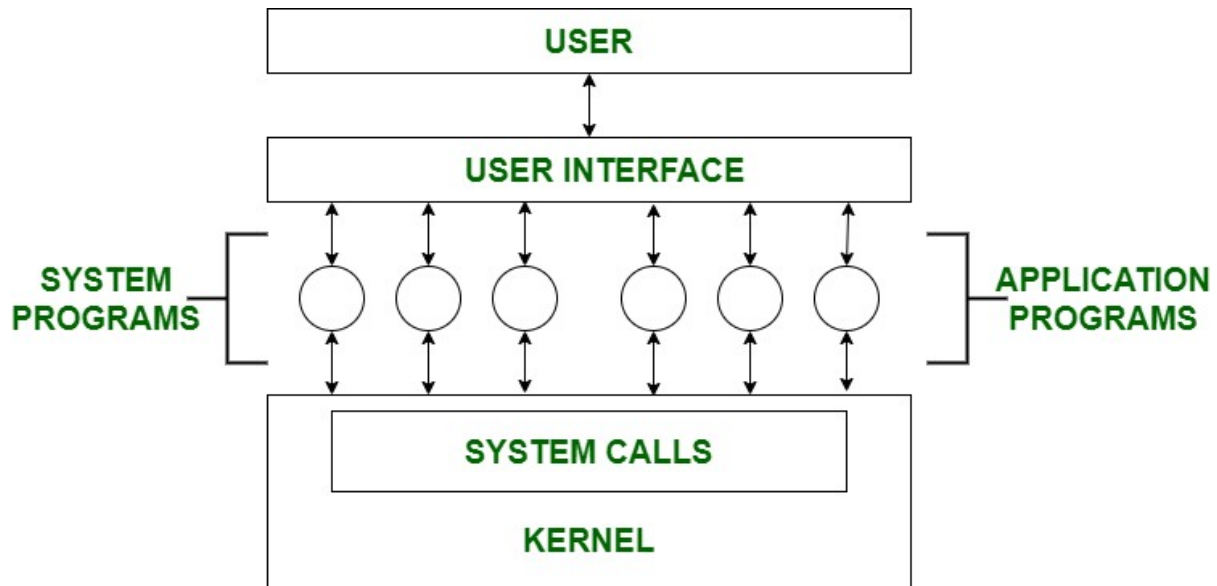
These operating systems are mainly for mobile devices. Examples of such operating systems are Android and iOS.

Introduction of System Call

A system call is a programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on. A system call is a way for programs to **interact with the operating system**. A computer program makes a system call when it requests the operating system's kernel. System call **provides** the services of the operating system to the user programs via the Application Program Interface (API). System calls are the only entry points into the kernel system and are executed in kernel mode.



- A user program can interact with the operating system using a system call. A number of services are requested by the program, and the OS responds by launching a number of systems calls to fulfill the request.
- A system call can be written in high-level languages like C or Pascal or in assembly language. If a high-level language is used, the operating system may directly invoke system calls, which are predefined functions.
- A system call is initiated by the program executing a specific instruction, which triggers a switch to kernel mode, allowing the program to request a service from the OS. The OS then handles the request, performs the necessary operations, and returns the result back to the program.
- System calls are essential for the proper functioning of an operating system, as they provide a standardized way for programs to access system resources. Without system calls, each program would need to implement its methods for accessing hardware and system services, leading to inconsistent and error-prone behavior.



Services Provided by System Calls/ Types of System Calls

- Process Creation and Management
- Main Memory Management
- File Access, Directory, and File System Management
- Device Handling(I/O)
- Protection
- Networking, etc.
 - **Process Control:** end, abort, create, terminate, allocate, and free memory.
 - **File Management:** create, open, close, delete, read files, etc.
 - **Device Management**
 - **Information Maintenance**
 - **Communication**

Similar types of operations are grouped into one single system call category. System calls are classified into the following categories:



Examples of a System Call in Windows and Unix

Process	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	Fork() Exit() Wait()
File manipulation	CreateFile() ReadFile() WriteFile()	Open() Read() Write() Close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() Read() Write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	Getpid() Alarm() Sleep()

Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	Pipe() Shmget() Mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorgroup()	Chmod() Umask() Chown()

System Programs in Operating System

System programs in an operating system are software tools that help users manage files, run programs, and control system resources. They include file managers, program loaders, compilers, and system utilities, making it easier to operate the computer efficiently and keep it running smoothly.

Here are the **examples of System Programs**:

1. **File Management:** File management is defined as the process of manipulating files in the computer system; its management includes the process of creating, modifying and deleting files.
2. **Command Line Interface (CLI's):** CLIs is the essential tool for user. It provides user facility to write commands directly to the system for performing any operation. It is a text-based way to interact with operating system. CLIs can perform many tasks like file manipulation, system configuration and etc.
3. **Device drivers:** Device drivers work as a simple translator for OS and devices. Basically it act as an intermediary between the OS and devices and provide facility to both OS and devices to understand each other's language so that they can work together efficiently without interrupt.
4. **Status Information:** Information like date, time, amount of available memory, or disk space is asked by some users. Others are providing detailed performance, logging, and debugging information which is more complex. All this information is formatted and displayed on output devices or printed. Terminal or other output devices or files or a window of GUI is used for showing the output of programs.
5. **Programming-Language support:** For common programming languages, we use Compilers, Assemblers, Debuggers, and interpreters which are already provided to users. It provides all support to users.

6. **Program Loading and Execution:** When the program is ready after Assembling and compilation, it must be loaded into memory for execution. A loader is part of an operating system that is responsible for loading programs and libraries. It is one of the essential stages for starting a program. Loaders, relocatable loaders, linkage editors, and Overlay loaders are provided by the system.
7. **Communications:** Connections among processes, users, and computer systems are provided by programs. Users can send messages to another user on their screen; User can send e-mail, browsing on web pages, remote login, and the transformation of files from one user to another.